Databases

Introduction to Software Systems

SQL & NoSQL

01

Datastores

The memory of the internet. Without databases, web applications would forget everything the moment you refresh the page.

What is a Database?

- **Persistent Memory**

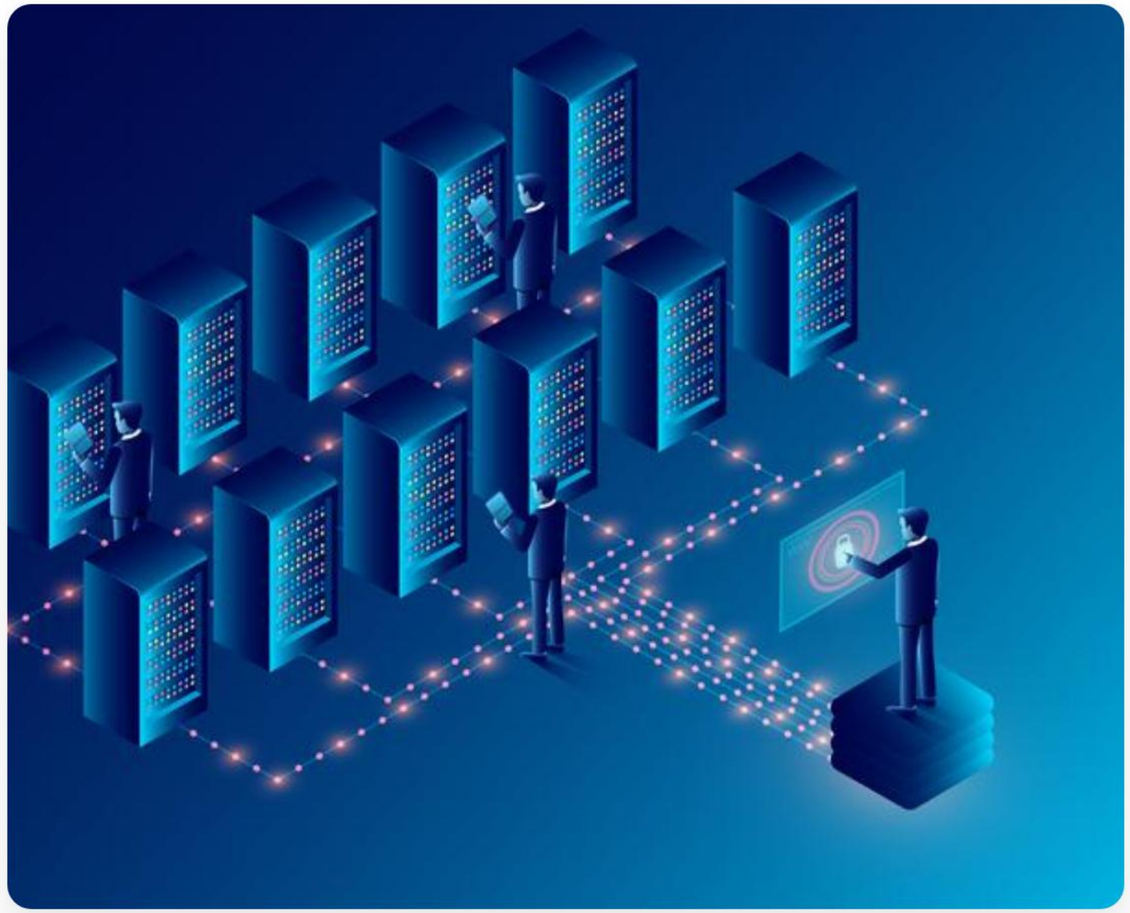
  Unlike standard variables in your code (which are lost on refresh), databases store data permanently on a hard drive.

- **Database Management System (DBMS)**

  The specialized software that interacts with end-users, applications, and the database itself to capture and analyze data safely and efficiently.

- **Scale and Speed**

  Designed to search through millions of records in milliseconds, handling thousands of concurrent users.



The Great Divide: SQL vs. NoSQL

SQL (Relational)

Data is organized into highly structured **Tables** (rows and columns). Like a strict Excel spreadsheet.

- **Examples:**
  MySQL, PostgreSQL
- **Schema:**
  Rigid and predefined
- **Best For:**
  Complex queries, financial systems.

NoSQL (Non-Relational)

Data is stored in flexible formats like **JSON Documents**. It adapts dynamically to the shape of your data.

- **Examples:**
  MongoDB, Redis
- **Schema:**
  Dynamic and flexible
- **Best For:**
  Rapid development, unstructured data.

02

MySQL

The world's most popular open-source relational database. Let's learn to speak SQL (Structured Query Language).

Categories of SQL

DDL (Define)

Data Definition Language

Used to define the database structure or schema. Deals with creating, altering, and dropping tables.

CREATE, ALTER, DROP

DML (Manipulate)

Data Manipulation Language

Used for managing data within schema objects. This is how you insert, update, or remove rows.

INSERT, UPDATE, DELETE

DQL (Query)

Data Query Language

Used to retrieve data from the database. The most frequently used part of SQL.

SELECT

DDL: Shaping the Database

CREATE TABLE

Defines a new table and specifies the data types for each column.

```
CREATE TABLE Users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL,
  created_at TIMESTAMP
);
```

ALTER & DROP

Modify existing tables or completely destroy them.

```
-- Add a new column
ALTER TABLE Users
ADD email VARCHAR(100);

-- Delete table and ALL data
DROP TABLE Users;
```

DML: Manipulating Data

Adding Data

Use `INSERT INTO` to add new rows to your table.

```
INSERT INTO Users (username)
VALUES ('johndoe');

-- Insert multiple rows
INSERT INTO Users (username)
VALUES ('alice'), ('bob');
```

Modifying & Removing

Always use WHERE! Otherwise, you will update/delete *every* row.

```
UPDATE Users
SET email = 'new@email.com'
WHERE username = 'johndoe';

DELETE FROM Users
WHERE id = 1;
```

The Mighty SELECT

- **Retrieving Data**

  SELECT allows you to fetch exactly the data you need, filtering it down to the exact columns and rows.

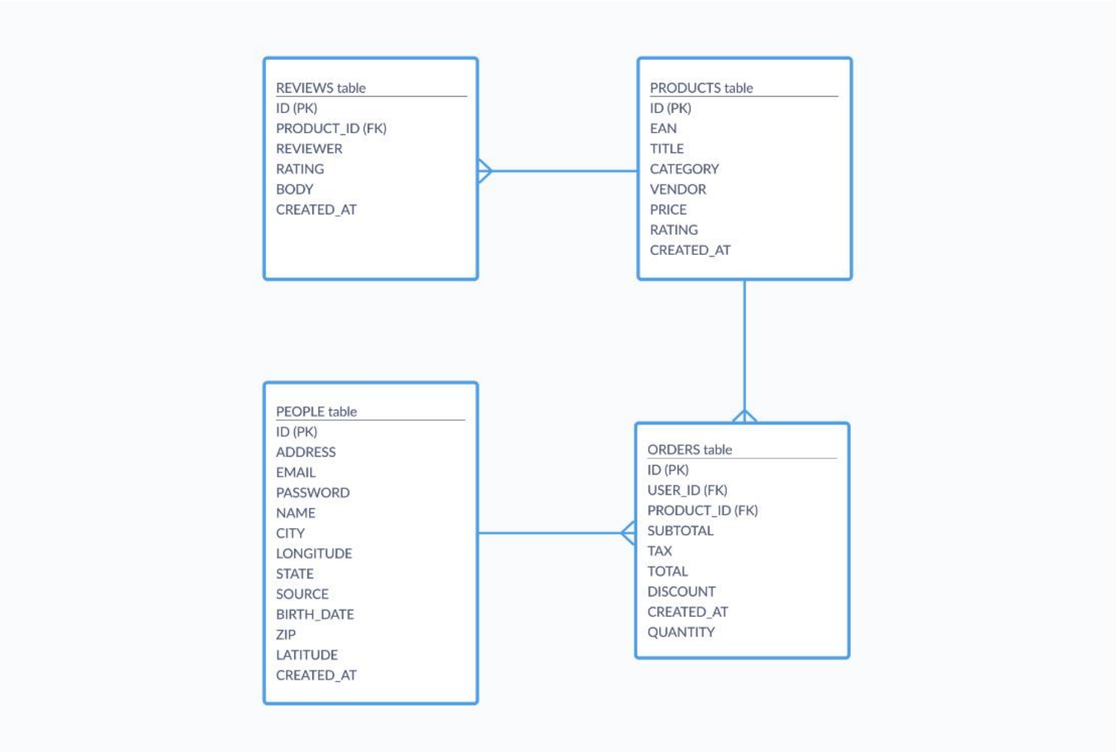
- **Filtering with WHERE**

  Use conditions like >, <, =, !=, or LIKE (for pattern matching) to filter rows.

- **Ordering & Limiting**

  Sort your results using ORDER BY and restrict the number of results using LIMIT.

```
SELECT id, username FROM Users
WHERE created_at > '2024-01-01'
ORDER BY username ASC
LIMIT 10;
```



Advanced SELECT

Nested Queries (Subqueries)

A query inside a query! Use the result of one query as the condition for another.

```
-- Find searches made by 'johndoe'
SELECT query_text FROM Searches
WHERE user_id = (
  SELECT id FROM Users
  WHERE username = 'johndoe'
);
```

JOINs (Brief Intro)

The true power of relational databases. Combine rows from two or more tables based on a related column.

```
SELECT Users.username, Searches.query
FROM Users
JOIN Searches
  ON Users.id = Searches.user_id;
```

03

MongoDB

A leading NoSQL database. It stores data in flexible, JSON-like documents, meaning fields can vary from document to document.

What is MongoDB?

- **Documents, not Rows**

  Data is stored as BSON (Binary JSON) documents. If you know JavaScript Objects, you already understand MongoDB data.

- **Collections, not Tables**

  Documents are grouped into "Collections". A collection can hold documents with entirely different shapes.

- **Flexible Schema**

  You don't need to define the structure beforehand. Just insert a document, and MongoDB handles the rest.

Illustration of NoSQL JSON document store showing tree structure

MongoDB CRUD Operations

Create & Read

Insert JSON objects directly. Query them using key-value filters.

```
// Insert (CREATE)
db.users.insertOne({
  name: "Alice", age: 25
});

// Find (READ) - WHERE age > 20
db.users.find({ age: { $gt: 20 } });
```

Update & Delete

Modify existing documents using operators like $set.

```
// Update
db.users.updateOne(
  { name: "Alice" },
  { $set: { age: 26 } }
);

// Delete
db.users.deleteOne({ name: "Alice" });
```

04

Integration

Connecting your application code to the database. We will look at Python for MySQL and Node.js for MongoDB.

Python + MySQL

Using Connectors

Using the `mysql-connector-python` library, you execute raw SQL strings directly from your Python script.

- **1. Connect:**
  Open DB channel
- **2. Execute:**
  Run a SQL command
- **3. Read:**
  Fetch the results

Example Implementation

```python
import mysql.connector

# 1. Connect to DB
db = mysql.connector.connect(
  host="localhost", user="root",
  password="pwd", database="test_db"
)

# 2. Execute SQL
cursor = db.cursor()
cursor.execute("SELECT * FROM Users")

# 3. Read Results
for row in cursor.fetchall():
  print(row)
```

Node.js + MongoDB

Native JavaScript

Because MongoDB uses JSON, it integrates flawlessly with JS backends using the official `mongodb` driver.

- **Asynchronous:**
  Network calls take time. Always use `async/await`.
- **Direct Mapping:**
  JS Objects translate perfectly to Mongo Documents.

Example Implementation

```
const { MongoClient } = require('mongodb');
const client = new
MongoClient('mongodb://localhost');

async function run() {
  await client.connect();
  const db = client.db('test_db');

  // Insert natively using JS Objects
  await db.collection('users')
    .insertOne({ name: 'Alice' });
}
```

05

In-Class Assignment

Time to apply what you've learned. We are going to design the database architecture for a simple Blog platform using both SQL and NoSQL.

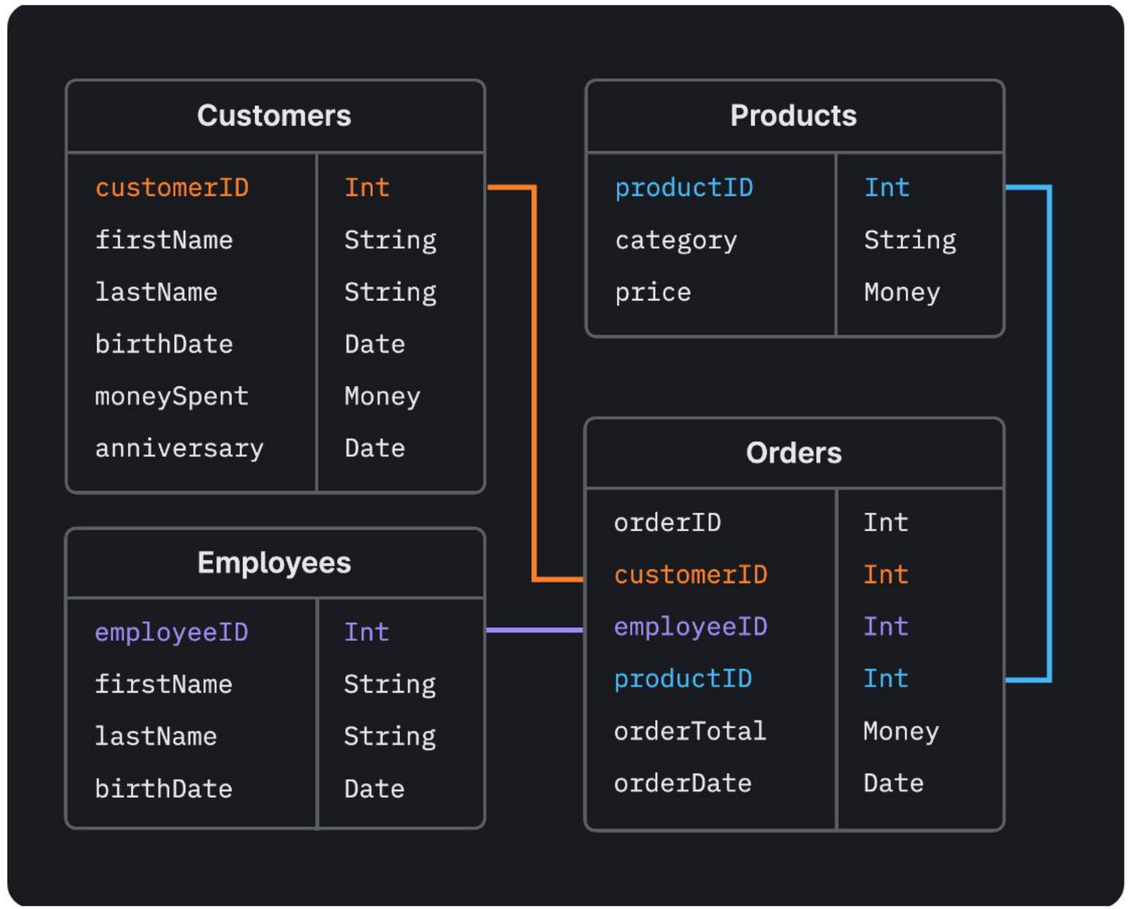
Task: The Blog Database

Scenario: You are building a new blogging platform. You need a backend to store Authors and the Posts they publish.

Step 1: Accept the **Lab 8 Assignment** link in the syllabus. This creates a brand new GitHub Classroom repository for you.

Step 2: Inside your new repo, write the SQL Schema to create two tables: `Authors` and `Posts`. Insert data and write a query to fetch an author's posts.

Step 3: Write the NoSQL equivalent. Design a JSON document representing a single Author who has an array of "posts" embedded directly inside their profile.



Activity Checklist

1. Setup Repo

Accept the Lab 8 GitHub Classroom link. Clone the new repository to your local machine to begin working.

2. SQL Schema

Create a `schema.sql` file. Write `CREATE TABLE` for `Authors` and `Posts`. Make sure `Posts` links to an author ID.

3. Mongo Model

Create a `mongo_model.json` file. Draft how you would embed a user's blog posts in a single NoSQL document.

Key Takeaways

Databases are Essential

They provide the persistent memory required for any modern dynamic web application to function.

SQL vs. NoSQL

SQL relies on rigid, relational tables (MySQL). NoSQL utilizes flexible, schema-less documents (MongoDB). Choose based on your data's shape.

The SQL Language

Mastering DDL (Create/Alter/Drop), DML (Insert/Update/Delete), and DQL (Select) gives you full control over relational data.

Integration is the Bridge

Drivers like `mysql-connector` and `mongodb` let backend languages (Python/JS) communicate directly with your datastore.

Open your terminal.

Accept the GitHub Classroom link and clone your new repo.

Remember to check your syntax carefully — SQL is very strict about missing semicolons!

Good luck with Lab 8!

Image Sources

Thumbnail for www.vecteezy.com

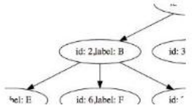
https://static.vecteezy.com/system/resources/previews/000/530/693/non_2x/data-center-server-room-cloud-storage-technology-vector.jpg

Source: www.vecteezy.com

---

https://www.metabase.com/learn/images/table-relationships/sample-database-erd.png

Source: www.metabase.com



https://i.sstatic.net/FhxZj.png

Source: stackoverflow.com

---

https://planetscale.com/assets/blog/content/schema-design-101-relational-databases/db72cc3ac506bec544588454972113c4dc3abe50-1953x1576.png

Source: planetscale.com